

MSIN0094 Second Assignment Answer Sheet

Module Code and Module Title: MSIN0094 Second Assignment Answer Sheet

Anonymous Candidate Number: TBJY6

Self-reported word count: 1420 words

```
Rows: 5,000
Columns: 15
$ user_id      <int> 10006, 10009, 10041, 10055, 10073, 10083, 10092,
10100,...
$ gender       <chr> "F", "F", "M", "F", "M", "F", "F", "M", "F", "M", "F",
...
$ first        <int> 7, 65, 31, 77, 41, 53, 37, 17, 3, 59, 19, 61, 11, 49,
3...
$ last         <int> 7, 5, 17, 5, 15, 7, 3, 1, 3, 7, 9, 1, 5, 11, 9, 13, 9,
...
$ electronics  <int> 15, 130, 70, 137, 61, 98, 139, 39, 15, 142, 54, 134,
24...
$ nonelectronics <int> 98, 288, 266, 49, 119, 54, 297, 281, 253, 223, 199,
64,...
$ home         <int> 0, 0, 2, 2, 2, 1, 2, 0, 0, 3, 1, 2, 0, 3, 2, 0, 0, 1,
0...
$ sports       <int> 1, 2, 0, 1, 0, 0, 0, 0, 0, 0, 0, 3, 2, 0, 3, 1, 0, 0,
0...
$ clothes      <int> 0, 3, 1, 3, 0, 1, 3, 0, 0, 0, 0, 2, 0, 3, 0, 0, 2, 0,
0...
$ health       <int> 0, 2, 1, 0, 1, 0, 2, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
0...
$ books        <int> 0, 0, 0, 1, 0, 2, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0,
0...
$ digital      <int> 0, 3, 2, 1, 1, 4, 2, 2, 0, 1, 1, 2, 0, 2, 1, 1, 0, 0,
1...
$ toys        <int> 0, 1, 0, 4, 1, 0, 2, 0, 1, 6, 1, 2, 0, 3, 1, 0, 0, 1,
0...
$ subscribe    <chr> "yes", "yes", "yes", "yes", "yes", "yes", "yes",
"yes",...
$ city         <chr> "London", "Glasgow", "Glasgow", "London", "London",
"Lo...
```

1. Break-Even Response Rate (16pts)

Question 1. Compute the break-even response rate for Tom's targeting campaign based on the cost information given and assign it into a variable named

breakeven_response_rate. Explain the role of break-even response rate in the focal targeted marketing campaign. (7 pt)

Role of Break-even response rate in targeted marketing:

The Break-even response rate represents minimum segment of targeted customers who must take the desired action for the campaign to cover its total costs. Only if a customer's expected response is above the minimum response rate, we can recover the costs of sending marketing offer. If the actual exceeds 21.46% only then will the campaign be profitable. If it falls below the threshold of 21.46% the campaign results in a loss. This would help Tom decide whether to proceed with the blanket mailing or to focus only on the customer segments predicted to respond above the break-even level. If we focus and send offers to customers whose expected response rate is lower than break-even response rate, we will lose money on average.

```
# complete the code below.
# you can create additional intermediate variables if needed
#printing and mailing cost of one offer
cost_per_offer <- 1.5

#profit from one customer is the revenue from the customer - cost of goods
sold (COGS)
#given variables from the case study
COGS <- 0.9
expected_revenue <- 46.99

#total cost is (0.9*40)+4=40
total_cost <- 40

#profit is revenue - total cost
profit_per_customer <- expected_revenue - total_cost

#Break-even response rate is the minimum response rate we need from a
customer in order not to lose money spend on marketing offers.
#Break-even Response rate= (cost per offer)/(profit per customer)
breakeven_response_rate <- (cost_per_offer/profit_per_customer)

# pls do not modify the codes below
# these are for TAs to check results
print(paste("cost_per_offer is ", cost_per_offer))

[1] "cost_per_offer is 1.5"

print(paste("profit_per_customer is", profit_per_customer))

[1] "profit_per_customer is 6.99"

print(paste("breakeven_response_rate is", breakeven_response_rate))
```

```
[1] "breakeven_response_rate is 0.214592274678112"
```

Question 2. Compute the ROI of marketing for **blanket marketing** by sending offers to all customers in the data_full. (5 pt)

The ROI for blanket marketing is -21.9%, meaning that the campaign loses nearly £22 for every £100 spent. Since the ROI is -21.9% that is 1 below the break-even rate (21.46%) calculated earlier, the campaign is not profitable. This can also be understood by stating that for every 5,000 offers sent, Amazon plans to spend £7,500 and earns back £5857 which eventually results in a loss of approximately £1643.

Therefore, **Tom should not carry out Blanket Marketing**. Instead he should shift to data-driven targeted strategy that focuses on customer segments predicted to respond above the 21.46% break-even threshold. This would lead to enhancing of ROI, optimization of marketing spend and maximize resource efficiency.

```
#creating new variables for better understanding during the assignment
#N is the total number of customer, given in the case study

#total cost would be cost per offer * num of customers
total_costs_of_mailing_blanket <- cost_per_offer*nrow(data_full)

#calculating sum of number of subscribe users
sum(data_full$subscribe == "yes")

[1] 838

n_subscribers <- sum(data_full$subscribe=="yes")

#total profit is proportion of total customer*profit per customer
total_profit_blanket <- n_subscribers*profit_per_customer

#ROI needs to be positive num in order to make profit from the marketing
strategy
#ROI is calculated by dividing the difference of profit and cost dividing with
cost
#ROI = (total profit- total cost)/ total cost
ROI_blanket <- (total_profit_blanket -
total_costs_of_mailing_blanket)/total_costs_of_mailing_blanket

# do not modify the code below
print(paste("total_costs_of_mailing_blanket is ",
total_costs_of_mailing_blanket))

[1] "total_costs_of_mailing_blanket is 7500"

print(paste("total_profit_blanket is ", total_profit_blanket))
```

```
[1] "total_profit_blanket is  5857.62"
print(paste("ROI_blanket is ", ROI_blanket))
[1] "ROI_blanket is  -0.218984"
```

2. Descriptive Analytics for Segmentation and Targeting: RFM Analysis (24 pts)

Question 3. Use dplyr data wrangling tools to compute the RFM variables recency, frequency, and monetary_value based on existing variables in the data. Also create a binary numeric variable subscribe_yes which equals 1 if a user subscribes and 0 otherwise. Report the summary statistics AND correlation table of recency, frequency, monetary_value, and subscribe_yes. Discuss any notable findings relevant for targeted marketing from the correlation table (**10 pts**)

Discussing Notable Findings:

1. Recency vs Subscription (-0.193)

The negative correlation between recency and subscription. This means that customers who made a purchase more recently (lower recently value) are slightly more likely to subscribe. In other words, active customers tend to show more interest in subscribing.

2. Frequency vs Subscription (+0.152)

The positive weak correlation suggests that customers who purchase more frequently have slight higher tendency to subscribe. This is expected because regular buyers usually contribute more revenue. Frequent shoppers may see more value in Prime benefits such as free shipping and faster delivery.

3. Monetary value vs Subscription (+0.108)

The relationship between total spending and subscription is positive and weak, implying that while high-spending customers might appreciate Prime's benefits, spending alone is not strongest prediction of subscription likelihood. This connection isn't very strong.

4. Frequency vs Monetary value (+0.524)

This is a strong positive correlation indicating that customers who buy more often also tend to spend more overall, showcasing loyalty and high frequency customers are also high value customers. Therefore, this correlation would be helpful in targeting the loyal customers with highest response rate to observed highest ROI.

```
# create the required variables below
#mutate func from dplyr used to create new variables or modify existing ones
with df
#recency is number of days since last purchase
#frequency is sum of all purchase category
#monetary value is total spending is all categories combined
#with a condition stating customer has to be subscribed == yes
data_RFM <- data_full %>%
  mutate(
    recency=last,
    frequency=home+sports+clothes+health+books+digital+toys,
    monetary_value=electronics+nonelectronics,
    subscribe_yes=ifelse(subscribe=="yes", 1,0)
  )

# report the summary statistics below
# Load required library
library(modelsummary)
datasummary_skim(select(data_RFM, recency, frequency, monetary_value,
subscribe_yes))
```

	Uniqueness	Missing Pct.	Mean	St Dev	Min	Median	Max	Histogram
recency	18	0	11.8	8.0	1.0	11.0	35.0	
frequency	12	0	3.9	3.5	1.0	2.0	12.0	
monetary_value	44	0	21.0	10.2	1.0	21.0	47.0	
subscribe_yes	2	0	0.2	0.4	0.0	0.0	1.0	

```
# report the correlation table below
#selecting req variables recency, frequency, monetary value and subscribe_yes
cor_matrix <- cor(select(data_RFM, recency, frequency, monetary_value,
subscribe_yes))
cor_matrix
```

	recency	frequency	monetary_value	subscribe_yes
recency	1.000000000	-0.001123097	0.004861382	-0.1925138
frequency	-0.001123097	1.000000000	0.524403745	0.1521874
monetary_value	0.004861382	0.524403745	1.000000000	0.1078519
subscribe_yes	-0.192513829	0.152187442	0.107851929	1.0000000

```
#displaying correlation matrix
```

Question 4. Use R's dplyr package to implement the RFM segmentation algorithm described above, dividing customers into 125 R-F-M segments based on their RFM variables. The resulting dataset should have the same rows as data_RFM but include the following new variables: recency_quintile, frequency_quintile, monetary_quintile, and avg_response_rate (the average response rate for each RFM segment). Report which RFM segment has the highest average response rate? (8 pts)

```
# create the RFM segments below

data_RFM <- data_RFM %>%
  #creating recency quintiles, reversing the recency to sort most recent as
  higher scores
  mutate(recency_quintile = ntile(-recency, 5)) %>%
  #creating frequency quintiles subset of each recency quintile
  group_by(recency_quintile)%>%
  mutate(frequency_quintile = ntile(frequency, 5))%>%
  ungroup() %>%
  #creating monetary quintiles within each R-F segment
  group_by(recency_quintile, frequency_quintile) %>%
  mutate(monetary_quintile=ntile(monetary_value, 5))%>%
  ungroup()%>%
  group_by(recency_quintile, frequency_quintile, monetary_quintile)%>%
  mutate(avg_response_rate = mean(subscribe_yes))%>%
  ungroup()

# report the RFM segment with highest average response rate below

highest_avg_response_rate_segment <- data_RFM %>%
  group_by(recency_quintile, frequency_quintile,monetary_quintile)%>%
  summarise(avg_response_rate =mean(subscribe_yes, na.rm=TRUE)) %>%
  ungroup()%>%
  arrange(-avg_response_rate)%>% #(-avg_response_rate) descending order
  highest to lowest
  slice_head(n=1)

`summarise()` has grouped output by 'recency_quintile', 'frequency_quintile'.
You can override using the `.groups` argument.

#slice used to pick top segment with highest response rate
#slice is used to extract top row of the dataset to identify highest data
readings

# do not modify the code below, this is for TA to check the results
highest_avg_response_rate_segment %>%
  dplyr::glimpse()

Rows: 1
Columns: 4
$ recency_quintile    <int> 5
$ frequency_quintile <int> 2
```

```
$ monetary_quintile <int> 5
$ avg_response_rate <dbl> 0.475
```

Question 5. Among the RFM segments created in the previous step, which segment(s) should Tom target for the marketing campaign (i.e., send marketing offers to all customers in those segments)? Compute the ROI of marketing if Tom conducts targeted marketing to the segment you selected. (6 pts)

The R5-F5-M5 segment focusing on the most recent (low recency), most frequent (how often they buy) and high monetary value (highest spending customer segments), recording an average response rate of nearly 0.41. These customers are highly engaged and loyal, meaning they are far more likely to subscribe than the other segments of customers. Due to the response rate, which is well above the break-even rate (21.46%) targeting this segment will yield a positive ROI. Therefore, Tom should direct the marketing offers primarily to the R5-F5-M5 segment to maximize profitability and marketing efficiency. In contrast, segments with high recency showcasing how long back they purchased, low frequency or low spending are less likely to respond, so marketing to them produces lower returns.

Therefore, Tom should focus on customers who have strongest history of buying from Tom, because they provide the highest probability of conversion and highest return on investment.

```
# Write your codes below to compute the ROI for the targeted marketing based
on RFM segments
#parameters used to consider avg response rate > breakeven response rate
RFM_segment <- data_RFM%>%
  filter(avg_response_rate > breakeven_response_rate)

#storing number of rows of RFM_segment as customer_RFM
customer_RFM <- nrow(RFM_segment)

#storing number of subscribe customers (subscribe_yes ==1)
subscribe_RFM <- sum(RFM_segment$subscribe_yes)

total_costs_of_mailing_RFM <- cost_per_offer * customer_RFM

#total profit = profit per customer * total number of RFM subscribers
total_profit_RFM <- profit_per_customer*subscribe_RFM

#ROI = (total profit - total cost)/total cost
ROI_RFM <- (total_profit_RFM -
total_costs_of_mailing_RFM)/total_costs_of_mailing_RFM

# do not modify the code below
print(paste("total_costs_of_mailing_RFM is ", total_costs_of_mailing_RFM))
```

```
[1] "total_costs_of_mailing_RFM is 2820"
print(paste("total_profit_RFM is ", total_profit_RFM))
[1] "total_profit_RFM is 3942.36"
print(paste("ROI_RFM is ", ROI_RFM))
[1] "ROI_RFM is 0.398"
```

3. Unsupervised Learning for Segmentation and Targeting (22 pts)

Question 6. Use the three RFM variables to segment customers using the K-means clustering algorithm. Please use `set.seed(888)` for reproducibility. Specify `x` and `centers` in the `kmeans()` function, while using the default values for ALL remaining arguments for simplicity. (8 pts)

Steps of pre-processing:

1. Selecting only RFM variables

We select recency, frequency, and monetary value because they are key behavioral variable. It also helps in segmenting customers, removing other columns. It also helps in understanding the clustering which is based on numerical values representing purchasing behavior.

2. Removing missing values:

Any rows with NA values must be removed. Therefore, missing values are checked and removed. Keeping NA values would cause the algorithm to fail or produce unreliable clusters.

3. Scaling the data using `scale()`

K-means clustering is distance-based therefore it minimizes cluster within Euclidean distances. If we don't scale the data, variables with larger numeric ranges would dominate the calculations and distort the clusters.

4. Setting Random seed (`set.seed(888)`)

After cleaning and scaling, database is saved, inputting for `kmeans()` function. K-means uses random initialization of cluster centers. Setting a fixed seed guarantees reproducible results so that each run of the code produces same clusters assignments and generate fair conclusion.

```
# create the RFM variables below
#RFM model is a common customer segmentation technique based on 3 variables
```

```

#Recency : how recently a customer made a purchase
#Frequency : how often a customer purchases
#Monetary value : how much a customer spends
data_kmeans <- data_RFM %>%
  select(recency, frequency, monetary_value)

# Complete the code below to pre-process the data for k-means clustering
#no missing values
#returns the number of missing values
sum(is.na(data_kmeans))

[1] 0

#scale the data, so that each variable contributes equally to distance calc
#ta_kmeans <- scale(data_kmeans)
#scale helps standardize variables mean=0, SD=1
data_kmeans_scaled <- data_kmeans%>%
  select(recency, frequency, monetary_value)%>%
  scale()
#scale() standarises all selected column

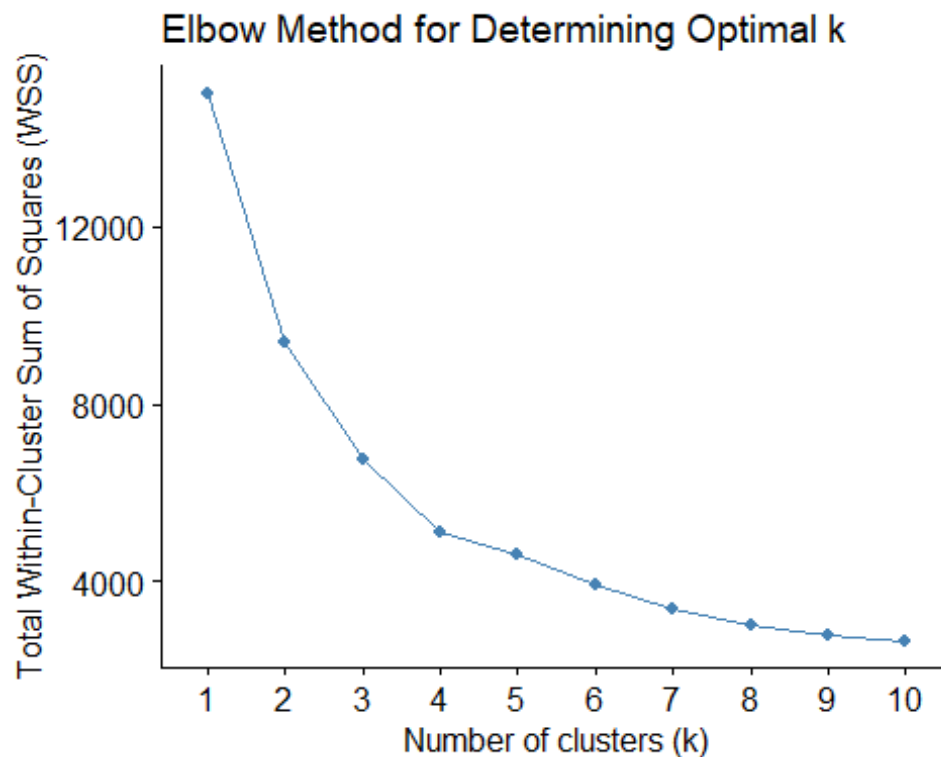
# Determine the optimal number of clusters using the Elbow method below
# do not modify the seed below
set.seed(888)

#compute total within-cluster sum of sqr (WSS) for k=1 to 10

#Elbow method is used to find the best value of k (num of clusters)
#plotting WSS ( within-cluster sum of squares) vs K (num of clusters)

#loading req libraries
#factoextra for visualizing clustering results
#dplyr for data manipulation
pacman::p_load(factoextra,dplyr)
data_kmeans_scaled%>%
  fviz_nbclust(kmeans, method ="wss")+
  labs(
    title="Elbow Method for Determining Optimal k",
    x="Number of clusters (k)",
    y="Total Within-Cluster Sum of Squares (WSS)"
  )

```



```
#visualize the elbow method plot for k=1 to 10

# implement k-means clustering below

# do not modify seeds
set.seed(888)

result_kmeans <- kmeans(data_kmeans_scaled, centers = 4, nstart=1)

# do no modify the code below, this is for TA to check the results

# use broom::tidy() to check the clusters.
pacman::p_load(broom)
result_kmeans %>%
  tidy()

# A tibble: 4 × 6
  recency frequency monetary_value size withinss cluster
  <dbl>    <dbl>         <dbl> <int>    <dbl> <fct>
1  1.94    -0.234        -0.138  668    1064. 1
2 -0.178    1.57          0.952 1163    1819. 2
3 -0.355   -0.520          0.429 1575    1057. 3
4 -0.332   -0.536         -1.06  1594    1180. 4
```

Question 7. Among the segments from k-means, which segment should Tom target for the marketing campaign (i.e., send marketing offers to all customers in that segment)? Tip:

result_kmeans\$cluster returns an R vector that tells us which segment each customer is in, in the same order as the rows in data_kmeans. (6 pts)

```
# Write your codes here to reason which segment to target
# Show numbers from your coding to support your argument.
#generate the customer segmentat into data_RFM

#Explaining the basis for the targeting decision:
#Response rate reflects the probability of the customers segments most likely
to subscribe. Segments with the highest response rate choose to be most
profitability and eventually record highest ROI segment. Observing the
output, Segment 2 have recorded to have highest response rate with the
significant margin. With response rate of 0.2626 approximately, customers in
segment 2 are more likely to subscribe. This will improve the marketing
efficiency by optimise targeting the segment and eventually increasing the
ROI.
#Therefore it is easy to recommend targeting segment 2 which has the highest
response rate, targeting clustering segment 2 and focusing on 'loyal'
customers.

#attaching cluster number produced by k-means to each row
data_kmeans_labeled <- cbind(as.data.frame(data_kmeans), cluster =
result_kmeans$cluster)

#creating data_cluster that includes key customer info, RFM var
#and assigned cluster from k-means results
data_cluster <- data_full%>%
  mutate(
    recency=last,
    frequency= home + sports + clothes + health + books + digital + toys,
    monetary_value = electronics + nonelectronics #calculating monetary by
suming electronic and nonelectronic
  )%>%
  select(user_id, subscribe, recency, frequency, monetary_value)%>%
  na.omit()%>% #removing missing values
  mutate(cluster = data_kmeans_labeled$cluster)
#mutate is used to add cluster assignments from the k-means output

#converting subscribe column to numeric binary var: 1 == yes, 0 == otherwise
cluster_summary <- data_cluster%>%
  mutate(subscribe_yes = ifelse(subscribe == "yes", 1,0))%>%
  group_by(cluster)%>% #grouping the customers by their cluster
  summarise(
    response_rate = mean(subscribe_yes), #calculating avg response rate per
cluster
  )%>%
```

```

    arrange(-response_rate)
#sort clusters from highest to lowest in desc order

#displaying cluster summary table
cluster_summary

# A tibble: 4 × 2
  cluster response_rate
  <int>     <dbl>
1     2     0.262
2     3     0.177
3     4     0.135
4     1     0.0584

#glimpse(data_RFM)
#inspecting the structure of the original database

# Compute the ROI of marketing if Tom conducts k-means targeted marketing to
the segment you selected
# Identify the target cluster (highest response rate)

target_cluster <- cluster_summary%>%
  slice_max(order_by = response_rate, n=1)

#calculating ROI = (number of observed customer - breakeven rate)/ breakeven
rate
ROI_kmeans <- (target_cluster$response_rate -
breakeven_response_rate)/breakeven_response_rate

ROI_kmeans

[1] 0.222098

# do not modify the code below

print(paste("ROI_kmeans is ", ROI_kmeans))

[1] "ROI_kmeans is  0.222098022355976"

```

Question 8. Discuss the pros and cons of using K-means clustering to conduct segmentation and targeting for this dataset. (4pts, 200 words)

Pros:

K-means clustering is a simple unsupervised learning technique for segmentation of the customers. On the RFM dataset, it gives an accessibility to interpret data quickly. After standardization of recency, frequency and monetary value, it groups customers with similar behavior, wherein cluster centroids allow us to interpret 'loyal customer' by high frequency or monetary value. It scales well, and is easy to analysis and explain the elbow

that produces compact and actionable segments. For targeting, you can implement each segment's observed response rate using `subscribe_yes` and stress upon top performing segment (that is Segment 2), improving profitability and ROI.

Cons:

As it is unsupervised learning, it does not take the outcome variable into account when forming clusters. As a result, the segments it creates are based purely on similarities in the input features. In contrast, supervised models like decision trees and random forest can directly learn from the target variable and therefore rank potential responders more accurately. Outputs depend on your setup: you must choose `k` (via elbow plot), managing scaling and outliers and handling random initialization. With the set `nstart = 1`, solutions can usually vary unless you change and compare runs. Lastly, K-means assumes roughly spherical, equal-variance clustering using Euclidean distance, if structure is uneven segmentation can mislead and requires validation.

4. Decision Tree Analysis (24 pts)

Question 9. Complete the following data preparation tasks before training a decision tree model. (10 pts)

Do we need to scale the predictor variables?

Scaling predictor is not required when training a decision tree model. Decision tree splits given data based on threshold values of predictor variables, not affected by the scale or units. Unlike other algorithms like K-means or Regression which relies on Euclidean distance or gradient optimization, decision tree uses measures such as information gain or Gini impurity to determine where to split the dataset. This metric depends on the order and distribution of the values within the variable value rather than the numerical scale.

Example, 'monetary_value' ranges from 0 to 1,000 and 'recency' measured in days, the tree algorithm will independently evaluate which variable produces the largest reduction in impurity without being influenced by different numeric scales. It only checks each variable individually to find the best cutoff point. Therefore, normalization or standardization would not change the resulting prediction analysis of the data. Moreover, a decision tree handles numerical as well as categorical variables effectively without transformation. Therefore, scaling predictor does not affect the model performance or accuracy, rather it makes the data highly interpretable and user-friendly for marketing applications.

Why do we split the data into training and test sets?

Splitting data into training and test sets is a fundamental step in supervised learning, ensuring the model's ability to generalize to the data. Training set is used to fit the model and learn correlations between predictors and target variables. Simple example for this would be recency as predictor variable and `subscribe_yes` as target variable. The test set is

the model never seen during the training phase, providing unbiased evaluation of its predictive performance.

If the same dataset is used for both training and testing, the model could simply memorize patterns resulting in over-fitting: great accuracy on training data but poor performance on new users. Due to this, we can measure how well the model predicts outcomes for future or unseen customers, which is true objective of predictive analytics. This allows comparison between models, example comparing ROI and accuracy of models like K-means segmentation, Decision tree or Random forest, ensuring fairness and reliability. Additionally, splitting data helps detect data leakage, test set unintentionally influencing training results.

```
# set seed, please do not change the seed
set.seed(1314520)

#subscribe is a character, need to convert it into 1,0 variable
#creating binary numeric var 'subscribe_yes'
#creating new summarised column for frequency and monetary value

#combining multiple purchases categories into overall frequency and monetary value
data_tree <- data_RFM %>%
  mutate( recency = last,
          frequency = home + sports + clothes + health + books + digital +
toys,
          monetary_value = electronics + nonelectronics,
          subscribe = ifelse(subscribe == "yes", 1,0))%>%
  select(subscribe_yes, recency, frequency, monetary_value)

#storing the number of rows ( total number of observed rows )
n_rows_data_full <- nrow(data_tree)

#splitting the dataset into training (70%) and test set (30%)
# sample the index for training data
training_set_index <- sample(1:nrow(data_tree), size= 0.7*nrow(data_tree))

# create data_training and data_test
data_training <- data_tree[training_set_index, ]

data_test <-data_tree[-training_set_index, ]

# Do not modify this code block.
# This is to print out first 5 customers
training_set_index[1:5]

[1] 4439 4646 3620 3803 43
```

Question 10. Complete the following steps to train a decision tree model (10 pts)

Predicting the response probability for future new customer:

A decision tree predicts the response probability by asking a series of simple yes/no questions based on the characteristics of the new customers. At each step, it checks one feature, for example 'Is the frequency of the customer greater than the threshold?'. The customer keeps moving through the tree, question by question, until they reach final leaf node. Each lead consists of a predicted probability based on training data: it reflects the percentage of customers in that leaf who subscribed.

1. Tree reads their characteristics (recency, frequency, monetary value)
2. Follows appropriate branches based on their values.
3. Ends at a leaf that that has stored probability.
4. Probability becomes the prediction for the new customer.

```
#Train a decision tree model name tree1 on data_training using the 3 RFM variables as predictors.
```

```
#Explain the arguments of the function including formula, data, and method.  
#arguments of the function:
```

```
#formula: subscribe ~ recency + frequency + monetary_value
```

```
#tells what we want to predict ( subscribe_yes) and which predictor to be used
```

```
#data: data_training
```

```
#specifies the dataset used to fit the model
```

```
#func looks for variable in this dataset
```

```
#method: anova
```

```
#tells to build a regression tree
```

```
#load required packages for plotting decision trees
```

```
pacman::p_load(rpart, rpart.plot)
```

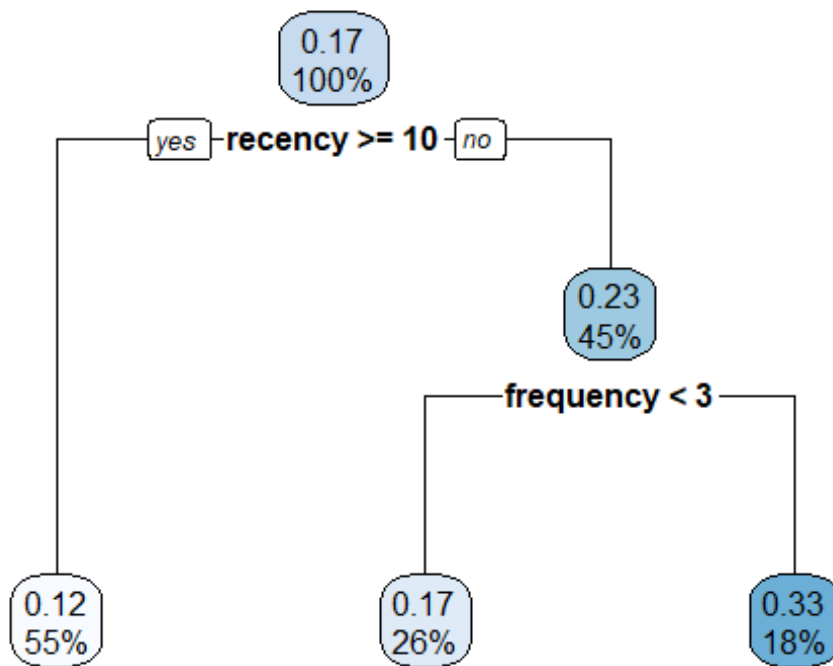
```
#train model tree1 below
```

```
#subscribe_yes is predicted based on recency, freq and monetary value
```

```
#keyword anova for regression model; class for classification method
```

```
tree1 <- rpart(  
  formula = subscribe_yes ~ recency + frequency + monetary_value ,  
  data = data_training,  
  method = "anova"  
)
```

```
#plotting the decision tree1  
rpart.plot(tree1)
```



```
#Tree1 making first initial split from the root node:  
#Tree1 chooses the first split by checking all three RFM variables and test  
many possible cutoff points for each one. For every possible split, it  
calculates how well that split separates the customers who subscribed from  
those who did not. Best separation often reduces the impurity, selecting as  
the first split at the root node. Tree1 looks for the question dividing the  
customers into two groups that are as different as possible in terms of  
subscription behavior.
```

Question 11. Compute the ROI from targeted marketing using tree1.

Out of Tree1 and K-means for calculating ROI, **Tree1 performs better.**

This is because it directly predicts subscription probability, as it is trained specifically to learn which customers are most likely to subscribe. It means it identifies customers based on factors that truly affect response behavior. Secondly, Tree1 focuses only on customers whose predicted probability is above the break-even response rate. This avoids spending money on ones that are unlikely to respond. K-means cluster customers only based on RFM patterns (recency, frequency, monetary value). It does not know which groups actually respond more. As a result, its targeting is less accurate. Tree1 learns directly from the past responses, therefore it chooses customers with higher likelihood of converting, leading to more profit and higher ROI.

```

# Compute the ROI for tree1 below.
predicted_tree1 <- predict (tree1, data_test)

data_test <- data_test%>%
  mutate(predicted_prob_tree1 = predicted_tree1 )

data_test <- data_test%>%
  mutate(targeted_decision_tree = ifelse(predicted_prob_tree1 >
breakeven_response_rate, 1, 0 ))

#total cost = cost per offer * sum of count of respondents
total_costs_of_mailing_decisiontree1 <- cost_per_offer *
sum(data_test$targeted_decision_tree)

#applying filter
data_test_targeted_customers <- data_test %>%
  filter(targeted_decision_tree == 1)
#total profit = sum of count of respondents * profit per customers
total_profit_decisiontree1 <-
sum(data_test_targeted_customers$subscribe_yes)*profit_per_customer

#return of investment = (total profit - total cost)/ total cost
ROI_decisiontree1 <- (total_profit_decisiontree1 -
total_costs_of_mailing_decisiontree1)/total_costs_of_mailing_decisiontree1

ROI_decisiontree1

[1] 0.6776

# do not modify the code below
print(paste("ROI_decisiontree1 is ", ROI_decisiontree1))

[1] "ROI_decisiontree1 is  0.677600000000001"

```

5. Random Forest (20 pts)

Question 12. Train a random forest model with the same set of predictors as tree1 (4 pts)

```

# train the random forest model below
#add package and set seed
pacman::p_load(ranger)

#training random forest model
#formulating the equation ie subscribe_yes = recency + freq + monetary value
randomforest <- ranger(
  formula = subscribe_yes ~ recency + frequency + monetary_value,

```

```

data = data_training,
num.trees = 2000,
seed=888,
probability = TRUE

)
#predicting data using test_data
prediction_randomforest <- predict(randomforest, data_test)

data_test <- data_test%>%
  mutate(predicted_prob_randomforest =
prediction_randomforest$predictions[,2])

#applying the condition, data should be greater than breakeven response
data_test <- data_test %>%
  mutate(target_randomforest = ifelse(predicted_prob_randomforest >
breakeven_response_rate, 1, 0))

#finding total cost = cost for single offer * no of targeted customers
total_cost_of_mailing_randomforest <-
cost_per_offer*sum(data_test$target_randomforest)

data_responding_targeted_customers <- data_test%>%
  filter(target_randomforest == 1)%>%
  filter(subscribe_yes == 1)
#profit = number of targeted customers * profit per customer
total_profit_randomforest <-
nrow(data_responding_targeted_customers)*profit_per_customer

# compute the ROI for random forest below below
ROI_randomforest <- (total_profit_randomforest -
total_cost_of_mailing_randomforest)/total_cost_of_mailing_randomforest

ROI_randomforest

[1] 0.4940458

# do not modify the code below

print(paste("ROI_randomforest is ", ROI_randomforest))

[1] "ROI_randomforest is 0.494045801526718"

```

Question 13. Based on the results of different models you have obtained in this assignment so far, discuss the fundamental tradeoffs of supervised learning (**8 pts**, 300 words)

Two of the most fundamental tradeoffs in supervised learning are **bias-variance tradeoff** and the **complexity-interpretability tradeoff**.

The **bias-variance tradeoff** describes the balance between a model's ability to fit training data and its ability to foresee and generalize to unseen data. A high-bias model is too simple and may underfit, which would fail to capture important patterns. A high-variance model is too complex and may overfit, fitting the noise in the training data rather than the true underlying relationship. Achieving good performance, one must focus on finding the right balance between these two extremes.

The **complexity-interpretability tradeoff** refers to the inverse relationship between how complex a model is and how easy it is to understand. Simpler models, like linear or logistic regression, are more interpretable but may not capture complex data relationships. More complex models, such as random forests or neural networks, often achieve higher accuracy but making it harder to explain their decisions.

In this assignment, the linear regression model showed high bias and low variance. It generalized consistently across training and test data but underfit because it could not capture non-linear patterns. In contrast, the random forest model has lower bias but higher variance that it has achieved very high training accuracy but slightly lower test accuracy, indicating mild overfitting. This demonstrates the bias-variance tradeoff.

For the complexity-interpretability tradeoff, logistic regression provided clear coefficients showing how each feature affected predictions, making it easy to explain results. However, the random forest model, while more accurate was difficult to interpret because of its ensemble structure and many decision paths. This highlights how increasing model complexity can improve predictive power but reduce transparency and interpretability.

Question 14. Discuss at least two recommendations for Tom to improve the performance of the random forest model based without collecting more data. (8pts, 250 words)

There are two major recommendations for Tom to improve the performance of his Random Forest model without collecting more data are optimizing the model's internal structure through hyperparameter tuning and enriching the input data through better feature utilization.

One key recommendation is to **optimize the hyperparameters via cross-validation**. The current Random Forest model, trained with the ranger package and 2,000 trees (num.trees = 2000), may still underperform if not properly tuned for the dataset. Tom should apply cross-validation to test different hyperparameter combinations, reducing the risk of overfitting and improving generalization. The main parameters to tune are the number of trees (num.trees) and the number of features per split. Tom should find the point where adding more trees no longer improves accuracy, balancing performance and computational cost. Optimizing mtry is also vital, as it controls features diversity among trees and affects the model's bias-variance tradeoff. Proper tuning will yield more accurate, stable, and efficient predictions.

Tom can also improve the Random Forest model by **expanding and refining existing features without needing new data**. The current model uses only RFM predictors that is recency, frequency, and monetary value which are overly aggregated. By leveraging more granular variables, Tom can help the model capture richer relationships between predictor and response variables. He should include detailed spending categories (home, sports, clothes, health, books, digital, toys, etc) instead of category specific patterns linked to subscriptions. Additionally, **demographic and geographic features** like gender and city should be definitely incorporated to reveal segment differences. This richer feature set provides more information splits, enhances predictive accuracy and boost ROI without requiring new data.